



Lecture 16: Mounting the Application

INTERVIEW QUESTION

[> CORE](#)

*You've written an `App.svelte` file.
How does it actually get onto the page?*

Svelte compiles your component to JavaScript, but how does that JavaScript end up rendering on the screen? What actually puts your component into the DOM?

Writing a Svelte component is only half the battle. Once your `.svelte` file is compiled into highly optimized JavaScript, you still need a way to take that code and inject it into the browser's Document Object Model (DOM) so the user can actually see it.

The Blank Canvas Problem

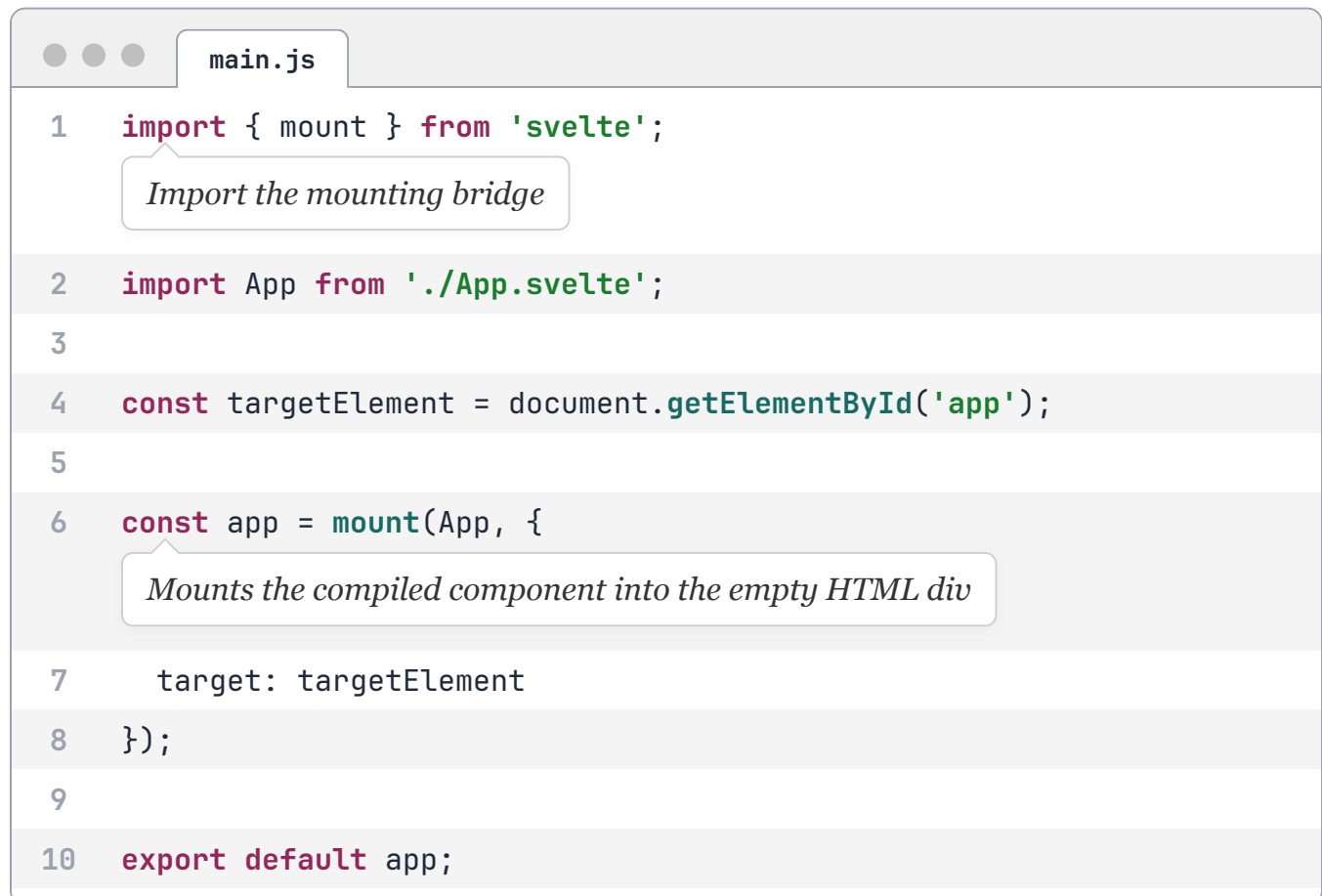
Imagine you have just finished writing the most beautiful `App.svelte` file ever created. You open up your project's `index.html` file in the browser, expecting to see your application, but you are greeted with a completely blank white screen.

If you inspect the HTML file, you'll see a single empty `<div>` with an id of `app`. Your component exists, but the browser has no idea what an `App.svelte` file is. Browsers only understand raw HTML, CSS, and JavaScript. You need a dedicated bridge to take the compiled JavaScript output of your Svelte component and physically append it to that empty `<div>`. If you skip this step, your app remains trapped as raw code in memory forever.

The `mount` Function

In Svelte 5, the bridge between your compiled components and the browser's DOM is the `mount` function.

This function takes two arguments: the root component you want to render (usually `App.svelte`), and the physical DOM element on the page where it should be injected (the target). This code typically lives in a single entry-point file, usually named `main.js` or `main.ts`.



```
1  import { mount } from 'svelte';
2  import App from './App.svelte';
3
4  const targetElement = document.getElementById('app');
5
6  const app = mount(App, {
7    target: targetElement
8  });
9
10 export default app;
```

Import the mounting bridge

Mounts the compiled component into the empty HTML div

The Initial Handshake

When you call `mount`, Svelte executes the `<script>` block of your `App.svelte` component for the very first time. It evaluates the initial values of your `$state` variables, computes any `$derived` values, and constructs the initial DOM nodes. It then grabs that newly constructed HTML and physically appends it as a child into your `target` element.

From that moment onward, the application is alive. The Svelte reactivity system takes over, and any further changes to the state will surgically update the DOM without ever needing to call `mount` again.

INTERVIEW TIP

To impress the interviewer: Note that in Svelte 5, `mount` completely replaces the old Svelte 4 class instantiation method (`new App({ target })`). The new functional `mount` API aligns perfectly with modern JavaScript patterns and makes the framework code significantly easier to tree-shake and optimize.

Summary

A `.svelte` file cannot render itself; it must be explicitly injected into an existing HTML document. To bring your application to life, you use the `mount` function exported by the `svelte` package. By providing your root component and a target DOM element (like an empty `<div>`), `mount` executes the component's setup logic, constructs the initial HTML, and permanently attaches your reactive application to the browser window.

	SVELTE 4 (Legacy)	SVELTE 5 (Modern)
API Style	Class instantiation (<code>new App()</code>)	Functional invocation (<code>mount()</code>)
Target Setup	Passed in options object	Passed in options object
Tree-shaking	Harder for bundlers to optimize	Excellent optimization